Support Control Tower: חלופה לפיתוח פנימי (בלי Spotfire)

תאריך	2026-10-12
בעלים	מנהל התמיכה הטכנית
מה זה	חלופה ל-MVP המקוצר (גרסה 2.0): אותו היקף ואותם נתונים, אבל אפליקציית וב שמפותחת בבית במקום Spotfire
מה לא משתנה	Salesforce נשארת מערכת התמיכה (קייסים, בקשות, SLA). ה-DW נשאר מקור הנתונים
החלטות שהתקבלו	אין כלי BI בתרחיש הזה. האנליטיקה תפותח כחלק מהמערכת. אחרי העלייה לאוויר: מפתח ו-DBA מתחזקים ומפתחים

0. תקציר והמלצה

ההיקף זהה ל-MVP המקוצר:

- שכבת נתונים מינימלית ב-DW.
- מסך חי לראשי הצוותים: P1 פתוחים, SLA בסיכון ובקשות לא משויכות.
- קישור מכל שורה ל-Salesforce.

ההבדל היחיד הוא מי מציג את המסך: במקום Spotfire, אפליקציית וב פנימית.

	Spotfire (MVP 2.0)	פיתוח פנימי (המסמך הזה)
זמן עד עלייה לאוויר	כ-4 שבועות	כ-7 שבועות
רענון "חי"	תלוי במה ש-Spotfire מאפשר (לאימות מול הספק)	בשליטה מלאה. יעד: עד 30 שניות, ואפשר להגיע לשניות
עלות ישירה	רישיונות ויועץ	זמן פיתוח, ובלי רישיונות
תחזוקה	הספק מתחזק את הפלטפורמה, ו-IT מתחזק את הניתוח	**הצוות הפנימי מתחזק הכול, לתמיד**: קוד, שרת, אבטחה ועדכונים
גלים הבאים (טרנדים, איכות לפי גרסה, בתי חולים)	קל: גרפים וסינונים מובנים	כל גרף הוא קוד שצריך לכתוב ולתחזק
אפליקציה לטלפון בהמשך	אפליקציה נפרדת	**אותו API משרת גם את אפליקציית הטלפון** (ראו מסמך MVP-Mobile)
אבטחה ו-HIPAA	הספק אחראי לפלטפורמה	הארגון אחראי לכל השכבות, כולל בדיקת חדירות

ההמלצה שלי, בהינתן שאין כלי BI והאנליטיקה תפותח בבית:

- **המסך החי מתאים לפיתוח פנימי מכל בחינה.** זה מסך תפעולי של שלושה מספרים ושתי רשימות, ושליטה מלאה ברענון שווה כאן יותר מכלי BI.
- **הגלים האנליטיים יעבדו בפיתוח פנימי רק אם מתכננים אותם כבר עכשיו.** בלי כלי BI, כל גרף, מסנן וייצוא נכתבים בקוד. כדי שזה לא יהפוך לפרויקט אינסופי, צריך שלושה דברים מהיום הראשון:

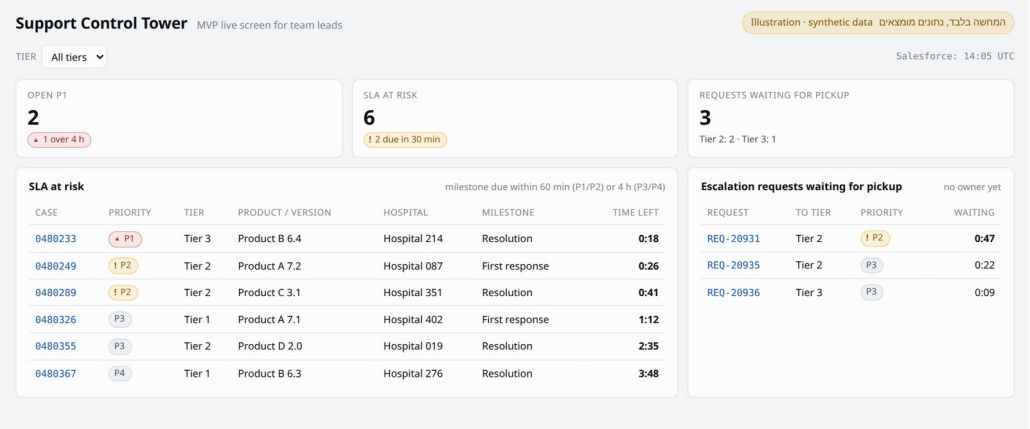
1. **הגדרות המדדים ב-DW ולא בקוד:** כל מדד הוא view, והאפליקציה רק מציגה אותו (סעיף 4.1).
2. **ספריית רכיבים חוזרים:** אריח, טבלה, גרף עמודות, גרף קו ומפת חום. כל מסך חדש מורכב מהם, ולא נכתב מאפס.
3. **ייצוא ל-CSV מכל טבלה:** בלי כלי BI, זו הדרך היחידה לענות על שאלה חד-פעמית בלי לפתוח משימת פיתוח.

- **"מפתח ו-DBA מתחזקים" זה בפועל פיתוח מתמשך, לא רק תחזוקה.** אם אותו מפתח גם מתחזק וגם בונה את הגלים האנליטיים, הוא צריך להיות במשרה מלאה על המערכת, לתקופה ארוכה (סעיף 7.4).

1. הבעיה והפתרון

הבעיה: ראשי הצוותים לא רואים במקום אחד מה דורש טיפול עכשיו: אילו קייסים קרובים להפרת SLA, אילו P1 פתוחים, ואילו בקשות אסקלציה עדיין בלי בעלים. כדי לדעת את זה היום צריך להריץ כמה דוחות ב-Salesforce.

הפתרון: אפליקציית וב פנימית עם מסך אחד, שמתעדכן לבד כל 30 שניות. היא נפתחת בכל דפדפן: במחשב, על מסך הקיר ובטלפון. כל שורה במסך נפתחת כרשומה ב-Salesforce.



איור: המסך החי (נתונים מומצאים)

2. קהל היעד (Early Adopters)

קבוצה	גודל	הרשאה
ראשי צוותים	7 (3 בטיר 1, 2 בטיר 2, 2 בטיר 3)	TeamLead: כל הנתונים במסך. בגרסה הזו אין במסך נתונים ברמת סוכן
מנהל התמיכה	1	Manager: הכול, כולל הגדרות
מסך בחדר התמיכה	1	Kiosk: קריאה בלבד, רק המסך החי (ראו 5.3)

3. הפיצ'רים ההכרחיים (Core Features)

#	פיצ'ר	מה בדיוק
F1	**שכבת נתונים מינימלית ב-DW**	views על רשומות פתוחות בלבד: `Case`, `CaseMilestone`, `Request`, ושמות מ-`Group`, `User` ו-`Account`. זהה ל-MVP 2.0
F2	**מסך חי**	שלושה אריחים: P1 פתוחים, SLA בסיכון ובקשות שממתינות לשיוך. שתי רשימות: SLA בסיכון (לפי הזמן שנשאר) ובקשות בלי בעלים (לפי זמן ההמתנה). לחיצה על אריח ה-P1 מציגה את רשימת ה-P1. מסנן לפי טיר. חותמת "עודכן לאחרונה"
F3	**קישור ל-Salesforce**	כל מספר קייס או בקשה הוא קישור לרשומה ב-Salesforce
F4	**כניסה עם SSO של החברה**	Entra ID עם MFA. בלי משתמש וסיסמה נפרדים
F5	**מצב מסך קיר**	תצוגה במסך מלא, בלי תפריטים, עם גופן גדול וערנון אוטומטי

ההגדרות (P1 פתוח, SLA בסיכון, בקשה ממתינה לשיוך, טיר) זהות ל-MVP 2.0, וממומשות ב-views ב-DW. כך אם בעתיד יחזרו ל-Spotfire או יוסיפו אפליקציה לטלפון, המספרים יישארו זהים.

בחוץ, בכוונה: בדיוק כמו ב-MVP 2.0: התראת P1, עומס לסוכן, דירוגים, R&D, RingCentral, דוחות PDF ויעדים.

4. ארכיטקטורה

```
Salesforce ──(real-time)──> DW: views on open records (Case, CaseMilestone, Request, names)
                                        │  read-only, Managed Identity
                                        ▼
                                Live API (Azure App Service)
                                 - queries the views every 15 s, keeps one snapshot in memory
                                 - Entra ID token check, roles: Manager / TeamLead / Kiosk
                                 - audit log
                                        │  HTTPS, JSON
                                        ▼
                                Web app (React + TypeScript)
                                 - polls the API every 30 s, or gets a push (phase 2: SignalR)
                                 - desktop, wall screen, phone browser
                                 - every row links to the Salesforce record
```

למה ככה:

- **השרת שואל את ה-DW כל 15 שניות, ושומר תמונת מצב אחת בזיכרון.** כל המשתמשים מקבלים את אותה תמונה. כך, גם אם 50 אנשים יפתחו את המסך, ה-DW מקבל שאילתה אחת כל 15 שניות ולא 50.
- **הדפדפן שואל את השרת כל 30 שניות.** זה פשוט ואמין, ומספיק ליעד של עד 60 שניות. אם בעתיד יידרשו שניות בודדות, אפשר להוסיף push מהשרת (Azure SignalR) בלי לשנות את שאר המערכת.
- **הפער בפועל** הוא הפער של ה-DW, ועוד עד 15 שניות בשרת, ועוד עד 30 שניות בדפדפן. כלומר עד כ-45 שניות, בתוספת הפער של ה-DW.

רכיב	בחירה מומלצת	למה
ממשק	React ו-TypeScript (Vite)	נפוץ, קל לגייס אליו, והקוד יכול לשמש גם בהמשך
שרת	ASP.NET Core (C#) או Node.js על Azure App Service	לבחור לפי מה שצוות IT כבר מכיר. Managed Identity לגישה ל-DW, בלי סיסמאות בקוד
הזדהות	Microsoft Entra ID (MSAL), עם תפקידים (app roles)	SSO של החברה, MFA ו-Conditional Access
סודות והגדרות	Azure Key Vault	
ניטור	Application Insights	זמני תגובה, שגיאות, והתראה ל-IT אם הנתונים ישנים
CI/CD	פייפליין אוטומטי, עם סביבת בדיקות וסביבת ייצור	כל שינוי נבדק לפני שהוא עולה

ה-API (חוזה ראשוני):

Endpoint	מה הוא מחזיר
`GET /api/me`	שם, תפקיד וטיר של המשתמש
`GET /api/live`	שלושת האריחים, שתי הרשימות וזמן העדכון האחרון (עם סינון אופציונלי לפי טיר)
`GET /api/live/p1`	רשימת ה-P1 הפתוחים (עם סינון אופציונלי לפי טיר)
`GET /api/health`	מצב השרת וגיל הנתונים (לניטור)

זה אותו API שתוכנן לאפליקציית הטלפון (MVP-Mobile). אם בהמשך בונים גם אותה, השרת כבר קיים.

4.1 תכנון לאנליטיקה בלי כלי BI

ה-MVP כולל רק את המסך החי, אבל כמה החלטות צריך לקבל כבר בו, כדי שהגלים הבאים לא ידרשו לבנות מחדש:

החלטה	מה עושים כבר ב-MVP	למה
שכבת	כל מדד מוגדר ב-view עם שם קבוע (למשל	שינוי הגדרה הוא שינוי ב-view אחד, בלי לגעת בקוד.

החלטה	מה עושים כבר ב-MVP	למה
מדדים ב-DW	`support.v_live_sla_at_risk`), ומתועד מול מילון המדדים	ה-DBA יכול לתחזק את ההגדרות לבד
ספריית רכיבים	האריחים והטבלאות של המסך החי נבנים כרכיבים כלליים, עם ספריית גרפים קיימת (למשל Apache ECharts)	כל מסך אנליטי בהמשך מורכב מרכיבים קיימים
API לפי מסכים	כל מסך מקבל endpoint משלו, וכולם עוברים דרך אותה שכבת הרשאות	מסך חדש לא פותח פרצת הרשאות חדשה
מסננים משותפים	רכיב אחד לטווח תאריכים, טיר, מוצר וגרסה. ב-MVP מופעל רק מסנן הטיר	בגלים הבאים כל המסכים יקבלו את אותם מסננים
ייצוא	ייצוא CSV מכל טבלה, עם audit log, ורק של השדות שמוצגים במסך	מענה לשאלות חד-פעמיות, בלי שהנתונים "מתפזרים" בקבצים לא מבוקרים
היסטוריה	שמירת היסטוריית השדות של Case ו-Request ב-DW מתחילה עכשיו, למרות שה-MVP לא משתמש בה	כל הגל האנליטי תלוי בה, ו-Salesforce מוחקת אותה אחרי כ-18 חודשים

5. אבטחה ודרישות לא פונקציונליות

5.1 אבטחה ו-HIPAA

- **צמצום נתונים:** ה-API מחזיר רק את השדות שהמסך מציג: מספר קייס או בקשה, Priority, טיר, מוצר, גרסה, שם בית חולים, שם הבעלים וזמנים. בלי טקסט חופשי ובלי פרטי מטופלים.
- **הרשאות בשרת:** הסינון לפי תפקיד נעשה תמיד בשרת, לא בדפדפן.
- **תקשורת:** HTTPS בלבד, עם כותרות אבטחה (CSP, HSTS). אין שמירה של נתונים בדפדפן מעבר לזיכרון של הדף.
- **ביקורת:** audit log לכל קריאה: מי, מה ומתי.
- **בדיקות:** עמידה ב-OWASP ASVS (רמה 2), ובדיקת חדירות לפני העלייה לאוויר.
- **תשתית:** כל הרכיבים ב-Azure, בתוך ה-BAA של החברה.

5.2 דרישות לא פונקציונליות

דרישה	יעד
פער בין שינוי ב-Salesforce לבין המסך	עד 60 שניות ב-95% מהזמן
זמן תגובה של ה-API	p95 פחות מ-500 מילישניות (התשובה מגיעה מתמונת המצב בזיכרון)
טעינה ראשונה של המסך	פחות מ-3 שניות
זמינות	99.5%, 24/7 (התמיכה עובדת 24/7)
דפדפנים	Edge ו-Chrome בגרסה עדכנית, ו-Safari בטלפון
תצוגה	מחשב, מסך קיר (1080p ו-4K) וטלפון
נגישות	ניגודיות ומקלדת. כל מצב מסומן גם בטקסט ובסמל, לא רק בצבע

5.3 מסך הקיר

מסך קיר הוא בעיה: אין עליו אדם מחובר, ואסור להשאיר עליו משתמש אישי מחובר. **ההמלצה:**

- משתמש ייעודי מסוג Kiosk ב-Entra ID, שיכול לראות רק את המסך החי, ורק לקריאה.
- Conditional Access שמגביל את המשתמש הזה למכשיר אחד מנוהל, ברשת של המשרד.
- ניתוק אוטומטי והתחברות מחדש פעם ביום.

6. מדדי הצלחה (KPIs)

אותם מדדים כמו ב-MVP 2.0, ושני מדדים שנוספו כי אנחנו מפעילים את המערכת בעצמנו:

מדד	יעד
אמינות	פער 0 מול Salesforce ב-10 בדיקות מדגמיות
עדכניות	עד 60 שניות מ-Salesforce למסך, ב-95% מהזמן
שימוש	7 מתוך 7 ראשי הצוותים פותחים את המסך לפחות פעם ביום עבודה
השפעה	ירידה בהפרות SLA של P1 ו-P2, וירידה בזמן החציוני עד שיוך Request, ביחס ל-4 השבועות שלפני העלייה (צריך להפיק את נקודת הבסיס מראש)
זמינות	99.5% ומעלה בתקופת הניסוי
אבטחה	0 ממצאים ברמת חומרה גבוהה שנשארו פתוחים מבדיקת החדירות

7. לוח זמנים, צוות ותקציב

7.1 לוח זמנים: 7 שבועות עד העלייה לאוויר, ועוד 4 שבועות ניסוי

שבוע	מה קורה	מי
1	סביבות, ריפו ו-CI/CD; רישום האפליקציה ב-Entra ID; גישה לקריאה מה-DW; בדיקת הפער בפועל בין Salesforce ל-DW; הפקת נקודת הבסיס למדדים	מפתחים, IT, DBA
2	views ב-DW; API עם תמונת מצב בזיכרון, הרשאות ו-audit log	DBA, מפתח Backend
3-4	הממשק: אריחים, רשימות, מסנן, קישורים ל-Salesforce, מצב מסך קיר	מפתח Frontend
5	ניטור, הקשחת אבטחה, ובדיקות על מחשב, על מסך הקיר ובטלפון	מפתחים, QA
6	בדיקת חדירות, ובדיקות מול Salesforce עם ראשי הצוותים	ספק בדיקות, ראשי צוותים
7	תיקונים, תיעוד והעברה ל-IT, עלייה לאוויר	מפתחים, IT
8-11	ניסוי ומדידה	מנהל התמיכה

7.2 צוות

תפקיד	היקף
מפתח/ת Full-stack מוביל/ה	משרה מלאה, שבועות 1-7
מפתח/ת Frontend	משרה מלאה, שבועות 2-6
QA	חצי משרה, שבועות 4-7
DBA	כ-2-4 ימי עבודה (views)
IT / DevOps	כ-5 ימי עבודה (Entra ID, סביבות, ניטור, מסך הקיר)
עיצוב	כ-3 ימי עבודה (על בסיס האיור הקיים)
מנהל התמיכה (בעל המוצר)	כ-10%: החלטות ובדיקות קבלה

7.3 תקציב

רכיב	כמות משוערת	סכום
פיתוח, QA ועיצוב	כ-14-18 שבועות-אדם	לפי תעריף פנימי או של ספק
בדיקת חדירות חיצונית	בדיקה אחת	לפי הצעת מחיר
Azure: App Service, Key Vault, Application Insights	שימוש של 8-10 משתמשים	נמוך, לחישוב על ידי IT

רכיב	כמות משוערת	סכום
רישיונות	אין	0
תחזוקה שוטפת	מפתח ו-DBA: תיקונים, עדכוני ספריות ואבטחה, שינויים קטנים	**כ-0.2-0.3 משרת מפתח וכ-0.1 משרת DBA, באופן קבוע** (לא כולל פיתוח הגלים האנליטיים, ראו 7.4)

ההערכות הן שלי ונועדו לתת סדר גודל. צוות הפיתוח צריך לתת הערכה משלו אחרי שבוע 1.

7.4 מפת דרכים: הגלים האנליטיים בפיתוח פנימי

ההערכות כאן גסות, ונועדו לתכנון קיבולת בלבד. כל גל יקבל אפיון והערכה משלו לפני שמתחילים אותו.

גל	תוכן	הערכה (שבועות-מפתח)	הערכה (שבועות-DBA)
2א	שכבת היסטוריה ומדדים: פרקי זמן בכל סטטוס ובכל טיר, Assets, Case Type	1	3-4
2ב	עומס והספק: עומס משוקלל לסוכן, FCR, אסקלציה, עבודה לפי Case Type	3-4	1
2ג	איכות מוצר: תקלות ל-100 Assets לפי גרסה, באגים מאושרים, דפוסי מוצר	3-4	1-2
2ד	R&D: בקשות לטיר 4 לפי גיל, כניסות מול יציאות	2	0.5
2ה	בתי חולים: דירוג מוחלט ומנורמל	2	0.5
2ו	דוחות PDF מתוזמנים (יומי, שבועי, Management Review)	2-3	0.5
2ז	התראות (P1, SLA, בקשות לא משויכות), אם יוחלט להכניס אותן למערכת	2	0
סה"כ		**כ-15-18**	**כ-7-9**
3	אפליקציה לטלפון, מעל אותו API (לפי MVP-Mobile, בלי בניית שרת מחדש)	כ-12-16 (שני מפתחי מובייל)	0

מה זה אומר בפועל:

- **מפתח אחד** שגם מתחזק (20-30% מהזמן) וגם מפתח: הגל השני ייקח בערך **5-6 חודשים** אחרי העלייה לאוויר.
- **שני מפתחים:** בערך **2.5-3 חודשים**.
- **DBA:** כ-7-9 שבועות עבודה לאורך התקופה, כלומר בערך חצי משרה בחודשים האלה.
- **בלי כלי BI, כל שאלה אנליטית חדשה היא משימת פיתוח.** כדאי לקבוע מראש תהליך: בקשות נאספות, מתועדפות פעם בחודש, ונכנסות לגל הבא. בלי תהליך כזה, המפתח יעבוד רק על בקשות חד-פעמיות.

8. סיכונים

סיכון	סבירות	השפעה	מיטיגציה
אין מפתחים פנויים בזמן	בינונית	חוסמת	לבדוק זמינות לפני שמחליטים. החלופה: ספק חיצוני, עם דרישת העברת ידע ל-IT
תלות באדם אחד: מפתח יחיד שמכיר את המערכת עוזב או לא זמין	בינונית	גבוהה	תיעוד, code review על ידי מפתח נוסף בארגון, ומפתח גיבוי שמכיר את הקוד ברמה בסיסית
המפתח מתחזק וגם מפתח, ושני הדברים נתקעים	גבוהה	בינונית	הקצאה ברורה: 20-30% לתחזוקה, והשאר לגל הבא. תיעדוף חודשי של בקשות
זחילת היקף: כל בקשה חד-פעמית הופכת למסך חדש	גבוהה	גבוהה	ייצוא CSV לשאלות חד-פעמיות, ותיעדוף חודשי לבקשות למסכים חדשים (7.4)
מסך הקיר מחובר עם משתמש אישי	בינונית	בינונית	משתמש Kiosk ייעודי (5.3)

סיכון	סבירות	השפעה	מיטיגציה
		(אבטחה)	
ה-"DW "real time בפועל עם פער גדול	נמוכה-בינונית	בינונית	לבדוק בשבוע 1
ממצאים בבדיקת החדירות מעכבים את העלייה	בינונית	בינונית	הקשחה בשבוע 5, לפני הבדיקה ולא אחריה

9. שאלות להחלטה

#	שאלה	למה זה חשוב
Q1	יש בארגון מפתחים פנויים ל-7 שבועות, או שעובדים עם ספק?	קובע את לוח הזמנים ואת העלות
Q2	**הוחלט:** מפתח ו-DBA. **פתוח:** כמה מהזמן שלהם מוקצה למערכת, והאם יש מפתח גיבוי	ראו 7.4
Q3	באיזו טכנולוגיה ה-DW (Azure SQL, Synapse, Fabric, אחר)?	משפיע על ה-views ועל הגישה מהשרת
Q4	באיזו שפת פיתוח צוות IT עובד היום (C# או JavaScript)?	כדי ש-IT יוכל לתחזק את השרת
Q5	**הוחלט:** פיתוח פנימי. **פתוח:** האם רישיונות Microsoft 365 של החברה כבר כוללים Power BI (למשל במסגרת E5)?	אם כן, יש כלי BI בלי עלות נוספת, ואפשר לחסוך חלק גדול מהגל השני
Q6	האם אפליקציית הטלפון עדיין בתוכנית?	אם כן, אותו API משרת את שתיהן, וזה מחזק את הבחירה בפיתוח פנימי